EXPERIMENT – 3

AIM :

To create an Employee-Department-Project database schema, insert sample data containing at least 30 employees, 5 departments and 8 projects and execute queries using Selection, Projection, Aggregate Functions, GROUP BY, HAVING, CASE Expressions, and ORDER BY.

SOURCE CODE :

1.Create Database and Tables

```
CREATE DATABASE Company_DB;
USE Company_DB;

CREATE TABLE Department (
   Dept_ID INT PRIMARY KEY,
   Dept_Name VARCHAR(50) NOT NULL,
   Location VARCHAR(50)
);

CREATE TABLE Project (
   Project_ID INT PRIMARY KEY,
   Project_Name VARCHAR(100) NOT NULL,
   Budget DECIMAL(12,2),
   Dept_ID INT,
   FOREIGN KEY (Dept_ID)
   REFERENCES Department(Dept_ID)
);

CREATE TABLE Employee (
   Emp_ID INT PRIMARY KEY,
   Emp_Name VARCHAR(100) NOT NULL,
   Salary DECIMAL(10,2),
   Job_Title VARCHAR(50),
   Dept_ID INT,
   Project_ID INT,
   FOREIGN KEY (Dept_ID)
   REFERENCES Department(Dept_ID),
   FOREIGN KEY (Project_ID)
   REFERENCES Project(Project_ID)
);
```

2. Insert Department Data

```
INSERT INTO Department VALUES
(1, 'IT', 'Delhi'),
(2, 'HR', 'Mumbai'),
(3, 'Finance', 'Mumbai'),
(4, 'Marketing', 'Bangalore'),
(5, 'Operations', 'Pune');
```

3. Insert Employee Data

```
INSERT INTO Employee VALUES
(1, 'Ravi', 55000, 'Developer', 1, 101),
(2, 'Rohit', 60000, 'Developer', 1, 101),
```

(3, 'Priya', 58000, 'Tester', 1, 101),
(4, 'Ananya', 75000, 'Manager', 1, 102),
(5, 'Rohan', 62000, 'Developer', 1, 101),
(6, 'Karan', 52000, 'Tester', 1, 101),
(7, 'Neha', 45000, 'HR Executive', 2, 103),
(8, 'Pooja', 50000, 'HR Executive', 2, 103),
(9, 'Amit', 65000, 'HR Manager', 2, 103),
(10, 'Simran', 48000, 'Recruiter', 2, 103),
(11, 'Vikas', 47000, 'Recruiter', 2, 103),
(12, 'Isha', 53000, 'HR Executive', 2, 103),
(13, 'Arjun', 68000, 'Accountant', 3, 104),
(14, 'Sakshi', 70000, 'Financial Analyst', 3, 104),
(15, 'Nikhil', 62000, 'Accountant', 3, 104),
(16, 'Rimance', 65000, 'Manager', 3, 104),
(17, 'Varun', 47000, 'Accountant', 3, 104),
(18, 'Isha', 63000, 'Analyst', 3, 104),
(19, 'Aditya', 55000, 'Marketing Executive', 4, 105),
(20, 'Satya', 60000, 'Marketing Executive', 4, 105),
(21, 'Mohit', 70000, 'Marketing Head', 4, 106),
(22, 'Tanya', 48000, 'Sales Executive', 4, 106),
(23, 'Yash', 56000, 'Sales Executive', 4, 106),
(24, 'Riya', 65000, 'Marketing Executive', 4, 105),
(25, 'Sahil', 58000, 'Operations Executive', 5, 107),
(26, 'Ravi', 46000, 'Operations Executive', 5, 107),
(27, 'Manish', 75000, 'Operations Manager', 5, 108),
(28, 'Nisha', 50000, 'Inventory Executive', 5, 108),
(29, 'Deepak', 54000, 'Inventory Executive', 5, 108),
(30, 'Kunal', 49000, 'Operations Executive', 5, 106);

4. Insert Project Data

INSERT INTO Project VALUES
(101, 'Website Development', 500000, 1),
(102, 'Mobile Application', 700000, 1),
(103, 'Recruitment System', 300000, 2),
(104, 'Financial Analysis', 600000, 3),
(105, 'Digital Marketing', 400000, 4),
(106, 'Sales Management', 550000, 4),
(107, 'Supply Chain', 800000, 5),
(108, 'Inventory System', 450000, 5);

5. Selection

SELECT *
FROM Employee
WHERE Salary > 50000;

6. Projection

SELECT Emp_Name, Salary
FROM Employee;

7. COUNT()

```
SELECT COUNT(*) AS Total_Employee
FROM Employee;
```

8. SUM()

```
SELECT SUM(Salary) AS Total_Salary
FROM Employee;
```

9. AVG()

```
SELECT AVG(Salary) AS Average_Salary
FROM Employee;
```

10. MAX()

```
SELECT MAX(Salary) AS Maximum_Salary
FROM Employee;
```

11. MIN()

```
SELECT MIN(Salary) AS Minimum_Salary
FROM Employee;
```

12. GROUP BY

```
SELECT Dept_ID, COUNT(*) AS Employee_Count
FROM Employee
GROUP BY Dept_ID;
```

13. Average Salary by Department

```
SELECT Dept_ID, AVG(Salary) AS Average_Salary
FROM Employee
GROUP BY Dept_ID;
```

14. HAVING

```
SELECT Dept_ID, COUNT(*) AS Employee_Count
FROM Employee
GROUP BY Dept_ID
HAVING COUNT(*) > 5;
```

15. CASE Expression

```
SELECT Emp_Name,
      Salary,
      CASE
         WHEN Salary >= 70000 THEN 'High Salary'
         WHEN Salary >= 50000 THEN 'Medium Salary'
         ELSE 'Low Salary'
      END AS Salary_Category
FROM Employee;
```

16. ORDER BY — Ascending

```
SELECT Emp_Name, Salary
FROM Employee
ORDER BY Salary ASC;
```

17. ORDER BY — Descending

```
SELECT Emp_Name, Salary
FROM Employee
ORDER BY Salary DESC;
```

18. Combined Query

```
SELECT Dept_ID, AVG(Salary) AS Avg_Salary
FROM Employee
GROUP BY Dept_ID
HAVING AVG(Salary) > 55000
ORDER BY Avg_Salary DESC;
```

OUTPUT :

